A BUILD-ALONG COURSE

Building an ORM on tokio-postgres

Derive macros, a model layer and a typed query builder on tokio-postgres, built from scratch as shelf — for readers of Async PostgreSQL in Rust.

Contents

0 Introduction and setup 1

0.1 What an ORM does, and what shelf will not do 1

0.2 Where shelf sits 2

0.3 The finished API in ten lines 2

0.4 Three crates, and why 2

0.5 How this book follows the build 2

0.6 Setup 3

0.7 The first derive, end to end 3

The programs 4

Claims 4

Summary 4

Exercises 5

1 Mapping rows by hand 6

1.1 A trait of shelf's own 6

1.2 By name or by index 7

1.3 Three ways to fail 7

1.4 Nested structs 7

1.5 A fetch helper 7

1.6 Counting the boilerplate 7

The programs 8

Side by side 8

The data 8

In stockroom 8

Claims 8

When not to use a mapping trait 8

Exercises 8

2 Declarative macros 9

2.1 Matching tokens 9

2.2 A `model!` macro 9

2.3 Tuples, one line per arity 10

2.4 `$crate` and hygiene 10

2.5 Where declarative macros stop 10

The programs 10

Side by side 11

The data 11

Claims 11

When not to use `macro_rules!` 11

Exercises 11

3 The first derive macro 12

3.1 Proc-macro crates 12

3.2 Three crates 12

3.3 Parsing with syn 3 13
3.4 Generating with quote 13
3.5 Hygiene 13
3.6 Seeing the expansion 13
3.7 Errors as values 14
The programs 14
Side by side 14
In shelf 14
Claims 14
When not to write a derive 14
Exercises 14
4 Attributes, errors and generics 15
4.1 Helper attributes 15
4.2 Parsing with `parse_nested_meta` 15
4.3 The six attributes 16
4.4 Every error at once 16
4.5 UI tests 16
4.6 Generics 16
4.7 What darling would save 17
The programs 17
Side by side 17
In shelf 17
Claims 17
When not to use an attribute 17
Exercises 17
5 Checking a statement once, not every row 19
5.1 What a lookup by name costs, again 19
5.2 A statement knows its columns 19
5.3 Types checked once 20
5.4 What the derive generates 20
5.5 Measured 20
5.6 Where it is used 20
The programs 20
Side by side 21
In shelf 21
Claims 21
When not to resolve once 21
Exercises 21
6 Tables: the model derive 22
6.1 What a table adds 22
6.2 `#[derive(Table)]` 22
6.3 SQL at compile time 23
6.4 Reading 23
6.5 Metadata as consts, and where they stop 23
The programs 23
Side by side 23
The data 24
Claims 24
When not to use `Table` 24

Exercises 24

7 The executor and shelf's errors 25

7.1 Why not `GenericClient` 25
7.2 The `Executor` trait 25
7.3 Three ways to send a statement 26
7.4 Transactions 26
7.5 `shelf::Error` 26
The programs 26
Side by side 27
Claims 27
When not to use the executor 27
Exercises 27

8 Writing rows: insert, update, delete, upsert 28

8.1 `#[derive(Insert)]` 28
8.2 Database defaults 29
8.3 Update and delete 29
8.4 Upsert 29
8.5 Constraint errors 29
The programs 29
Side by side 29
In stockroom 29
The data 30
Claims 30
When not to use shelf's writes 30
Exercises 30

9 Partial updates and optimistic locking 31

9.1 Absent or null 31
9.2 Three designs 31
9.3 Version columns 32
9.4 `updated_at` 32
9.5 ETags in axum 32
The programs 32
Side by side 33
In stockroom 33
Claims 33
When not to use `Patch` 33
Exercises 33

10 A typed query builder 34

10.1 What to type 34
10.2 Column constants 34
10.3 Expressions 35
10.4 Owned parameters 35
10.5 Order, limit and pages 35
10.6 What fails to compile 35
10.7 The escape hatch 35
The programs 35
Side by side 36
In stockroom 36
Claims 36

When not to use the builder 36
Exercises 36
11 Relations without N+1 37
11.1 Declaring relations 37
11.2 N+1, measured 37
11.3 Batched loading 38
11.4 Pipelining independent loads 38
11.5 Joins 38
11.6 Why no lazy loading 38
The programs 38
Side by side 39
The data 39
Claims 39
When not to use relation loaders 39
Exercises 39
12 Bulk writes and streams 40
12.1 `insert_many` 40
12.2 Binary COPY from the derive 41
12.3 Measured 41
12.4 Streams of models 41
12.5 A COPY rejected at its start 41
The programs 41
Side by side 41
In stockroom 42
Claims 42
When not to use shelf's bulk writes 42
Exercises 42
13 Trusting the mapping: schema checks and tests 43
13.1 What can drift 43
13.2 `verify` 43
13.3 Where to run it 44
13.4 Testing a derive 44
13.5 Compile-time checking 44
The programs 44
Side by side 44
Claims 45
When not to use `verify` 45
Exercises 45
14 Capstone: stockroom on shelf 46
14.1 The shape 46
14.2 Models for a real schema 46
14.3 Line for line 47
14.4 Where shelf steps aside 47
14.5 What a production ORM would add 47
The programs 47
Side by side 47
The data 47
Claims 47
When not to use shelf 48

Exercises 48

Appendix — attribute reference, generated code and phrasebook 49

A Attribute reference 49

B What each derive generates 50

C Phrasebook 50

D Glossary 51

E Further reading 51

CHAPTER 0

Introduction and setup

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part I, first chapter. Builds on the whole of the main book, and on its Chapter 0 (the setup) and Appendix F (the crates built on tokio-postgres). §0.7 is already written, and its two programs run in `book.py --book orm check`. shelf: `#[derive(FromRow)]` and `fetch_all` (built, S1a); the API of §0.3 (mostly proposed). Crates: tokio-postgres 0.7.18, syn 3.0.9, prettyplease 0.3.0. Decisions: PLAN.md §10 (the name, three crates, the scope). Plan: orm/PLAN.md §1, §3, §4, §7.

Async PostgreSQL in Rust taught tokio-postgres as a driver: you wrote every SQL statement yourself, and turned every row into a struct with a `TryFrom<&Row>` that named each column again (Chapter 3). This book builds the layer that writes that code for you. **shelf** is a small ORM: derive macros that map rows to structs, a model layer that writes the everyday SQL, and a typed query builder. You build it one chapter at a time, and each chapter's code is in this repository, compiled and tested.

The goal is not to replace diesel or SeaORM. It is to know what an ORM does underneath, by building one on a driver you already understand: what a derive macro receives and returns, how it reports a mistake at the right line, what SQL a model method sends and how many round trips it costs, and where hand-written SQL is still the better tool.

In this chapter you will learn:

- what an ORM does for you, and the jobs shelf deliberately leaves to SQL
- where shelf sits among Rust's database libraries, and what each of them decides differently
- why shelf is three crates, and what each one holds
- how this book follows the build: stage programs, excerpts, and expansions printed by running the code generator
- how to set up the playground and run a chapter's examples
- what `#[derive(FromRow)]` writes, by running it

0.1 What an ORM does, and what shelf will not do

Plan:

- Three jobs: mapping rows to structs and back; writing the everyday SQL (find, insert, update, delete) from a struct; following relations between tables without one query per row.
- What shelf leaves out on purpose (PLAN.md §4.6), each with its reason: migrations (main book Chapter 14), checking queries against a live database at compile time, lazy loading and change tracking, several databases, a blocking API, and Postgres enums and composites (postgres-types' derives already map them).

- The thread through the book: every shelf method says what SQL it sends and how many round trips it costs, and every chapter ends with when not to use it.

0.2 Where shelf sits

Plan:

- One table, five columns at most: diesel and diesel-async (a schema DSL, compile-time checked), SeaORM (entities and `ActiveModel`, on sqlx), toasty (from the Tokio project, young), sqlx's `FromRow` (mapping only), postgres-from-row (a `FromRow` derive for tokio-postgres), Cornucopia (SQL first, Rust generated). The main book's Appendix F has the research; versions and dates from there, rechecked when the chapter is written (check).
- What shelf decides differently: runtime-checked mapping with one check per statement, SQL that a reader can predict, tokio-postgres underneath with nothing hidden.

0.3 The finished API in ten lines

Plan:

- A model with `#[derive(FromRow, Table)]`, a `New…` struct with `#[derive(Insert)]`, then insert, find, a filtered query and a batched relation load. Written as a runnable block once phase S1c exists; until then PLAN.md §4.4 shows its target shape.

0.4 Three crates, and why

Draft prose:

A derive macro is a function that the compiler calls with the tokens of your struct and that returns new tokens. Rust requires that function to live in a crate of its own kind, a *proc-macro crate*, which can export macros and nothing else. That is inconvenient: code that generates code is code you want to test and to show, and a proc-macro crate cannot be called by a test in another crate or by an example in this book.

So shelf is three crates. `shelf-codegen` is an ordinary library that holds everything the derives generate, as functions from a parsed struct to tokens. `shelf-macros` is the proc-macro crate, and is three lines per derive: parse, call `shelf-codegen`, return the result or the error. `shelf` is what you depend on: the traits and functions your code calls, and the derives, re-exported.

0.5 How this book follows the build

Plan:

- The problem: each chapter shows shelf at one stage, but the repository holds the finished crate (PLAN.md §3).
- Stage programs: each new mechanism first as a runnable single-file program, including the macro code, because syn and quote run in any program.
- Excerpts of the finished crate, kept true by `book.py excerpts`; what later chapters add is elided with `// …`.
- Nothing generated is typed by hand: expansions come from running shelf-codegen, compiler errors from the UI tests' `.stderr` files.
- The callouts: the main book's four, plus **Macro lens**.

0.6 Setup

Plan:

- The main book's setup (Chapter 0): PostgreSQL 18, `.env` with `ADMIN_DATABASE_URL`.
- The shelf playground, `orm/playground`: what it depends on, its `pretty` helper, and how a chapter's examples are generated and run (`python3 tools/book.py --book orm check --chapter N`).
- shelf's own tests (`cargo test -p shelf -p shelf-codegen`) and the database `shelf_test`.

0.7 The first derive, end to end

`#[derive(FromRow)]` reads each named field from the column of the same name. This program derives it for a struct and reads two rows into it with `shelf::fetch_all`:

```
use shelf::FromRow;

#[derive(Debug, FromRow)]
struct Product {
    id: i64,
    sku: String,
    price_cents: i64,
    tags: Option<Vec<String>>,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                sku text NOT NULL UNIQUE,
                price_cents bigint NOT NULL,
                tags text[]
            );
            INSERT INTO products (sku, price_cents, tags)
            VALUES ('MUG-01', 1299, '{kitchen}'), ('TEE-02', 2499, NULL);",
        )
        .await?;

    let products: Vec<Product> = shelf::fetch_all(
        &client,
        "SELECT id, sku, price_cents, tags FROM products ORDER BY id",
        &[],
    )
    .await?;

    println!("{products:#?}");
    assert_eq!(products.len(), 2);
    assert!(products[0].id < products[1].id);
    assert_eq!(products[0].sku, "MUG-01");
    assert_eq!(products[0].price_cents, 1299);
    assert_eq!(products[0].tags, Some(vec!["kitchen".to_string()]));
    assert_eq!(products[1].tags, None);
    Ok(())
}
```

`use shelf::FromRow` brings in two things with one name, the trait and the derive macro, because traits and macros live in different namespaces (serde's `Serialize` works the same way).

What did the derive write? A proc-macro crate can export only macros, so shelf keeps the code generation in an ordinary library, `shelf-codegen`, and this program can call it on a struct and print the result:

```
use syn::{DeriveInput, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product {
            id: i64,
            sku: String,
        }
    };
    let generated = shelf_codegen::from_row::expand(&input).expect("a struct with named
fields");
    let code = shelf_playground::pretty(generated);
    println!("{code}");
    assert!(code.contains(r#"column::<String>(columns, "Product", "sku", "sku")?"#));
}
```

It prints the `impl` the compiler sees in place of `#[derive(FromRow)]`. `resolve` finds and checks each field's column once per statement; `from_row_at` then reads each row by position (Chapter 5 explains why):

```
#[automatically_derived]
impl ::shelf::FromRow for Product {
    type Columns = (usize, usize);
    fn resolve(
        columns: &[::shelf::__private::tokio_postgres::Column],
    ) -> ::core::result::Result<Self::Columns, ::shelf::Error> {
        ::core::result::Result::Ok((
            ::shelf::__private::column::<i64>(columns, "Product", "id", "id")?,
            ::shelf::__private::column::<String>(columns, "Product", "sku", "sku")?,
        ))
    }
    fn from_row_at(
        row: &::shelf::__private::tokio_postgres::Row,
        columns: &Self::Columns,
    ) -> ::core::result::Result<Self, ::shelf::Error> {
        ::core::result::Result::Ok(Self {
            id: ::shelf::__private::get::<i64>(row, columns.0, "Product", "id")?,
            sku: ::shelf::__private::get::<String>(row, columns.1, "Product", "sku")?,
        })
    }
}
```

The programs

- Both programs in §0.7 exist and run (`run`): the first derives `FromRow` for a struct and reads two rows with `fetch_all`; the second runs `shelf_codegen::from_row::expand` on a struct and prints the expansion, which is pasted below it as captured.
- §0.3's program (`run`) waits for phase S1c.

Claims

- Docs: the crates' versions and dates in §0.2, from the main book's Appendix F (recheck).
- Opinion: an ORM is worth building once, to know what the one you use does for you.

Summary

Plan: one bullet per section.

Exercises

Plan:

1. Add a field to §0.7's `Product` that the table does not have, and read the error. *Hint:* it names the struct, the field and the column, before the query runs.
2. Rename a column with `AS` in the query and keep the program working without touching the struct; then the other way round. *Hint:* Chapter 4's `rename`.

> **Open questions.** (1) Whether §0.2's comparison table belongs here or in the appendix's phrasebook, with a short version here. (2) The ten-line taste needs S1b and S1c; until then, §0.3 can show only the mapping. (3) Whether to explain the absolute paths of the expansion (`::shelf`, `::core`) here or leave them to Chapter 3, which covers hygiene.

CHAPTER 1

Mapping rows by hand

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part I, second chapter. Builds on main book Chapter 3 (`Row`, `get` and `try_get`, NULL, `TryFrom<&Row>`) and Chapter 4 (what `query` sends). shelf: the first form of the `FromRow` trait, with `from_row` only, written by hand in the chapter's programs; shelf's own trait is built (S1a) and grows in Chapter 5. Crates: tokio-postgres 0.7.18. Findings: PLAN.md §12 #4 (how a name is looked up), #8 (what it costs). Plan: orm/PLAN.md §7, Chapter 1.

You have written this function before. Every struct that comes out of the database in stockroom has a `TryFrom<&Row>` that names each field and reads each column: `id: row.try_get("id")?`, `code: row.try_get("code")?`, and so on, once per field, for every struct. The SQL above it names the same columns again. Change a column and you change three places, and the compiler checks none of them against the table.

An ORM's first job is to write that function for you. Before writing the macro that does it, this chapter writes it by hand once more, carefully: as a trait of shelf's own, with the three ways it can fail, and with a measurement of what reading a column by name actually costs. The numbers decide the design of the derive two chapters from now.

In this chapter you will learn:

- why shelf has its own `FromRow` trait instead of using `TryFrom<&Row>`
- what `Row::try_get("sku")` does in tokio-postgres 0.7.18, and what it costs per row
- the three ways a mapping fails (a missing column, a wrong type, a NULL) and what each error says
- how to nest one struct inside another when both come from the same row
- a `fetch_all` helper that runs a query and maps its rows
- which parts of the hand-written code a macro should take over

---

1.1 A trait of shelf's own

Plan:

- `trait FromRow: Sized { fn from_row(row: &Row) -> Result<Self, Error>; }`, with `tokio_postgres::Error` as the error for now (shelf's own `Error` arrives with Chapter 4's better messages).
- Why not `TryFrom<&Row>`: it works (stockroom uses it), but a trait of shelf's own can grow associated items (Chapter 5's `Columns`), have impls for tuples (Chapter 2), and carry a name the derives can build on. The orphan rule is not the reason: both are fine for your own types.
- The first impl, by hand, for a `Product` with five fields.

1.2 By name or by index

Draft prose:

`row.try_get("sku")` looks like a map lookup, but a row has no map. In 0.7.18 it walks the row's columns and compares each name with `"sku"`; if none matches, it walks them again, ignoring ASCII case. Then, whether you asked by name or by position, it checks that the column's type can be read into the Rust type you asked for, and only then decodes the bytes. A twelve-field struct read by name therefore does up to twelve searches per row, each over up to twelve names.

Plan:

- The source, quoted from `row.rs` (`RowIndex for str`), with the FIXME about case.
- The measurement: 100,000 rows of 12 columns, read by name and by position (PLAN.md §12 #8: about 3× in a release build). The chapter's runnable version runs in a debug build, so it asserts only that by position is faster, and prints both times.
- What it means: the name lookup, not the decoding, dominates; the per-row type check costs nothing measurable.

1.3 Three ways to fail

Plan:

- A missing column ("invalid column `sku`"), a wrong type (the `WrongType` message), a NULL read into a non-`Option` field (`WasNull`): each provoked and its message captured from a real run, as tokio-postgres reports it.
- What those messages lack: the struct and the field. Chapter 4's errors add them.

1.4 Nested structs

Plan:

- A `Price { cents, currency }` inside a `Product`, both read from one row: the inner struct's `from_row` called from the outer one's.
- The catch: both use names from the same row, so the inner struct's columns must not clash with the outer's. Chapter 4's `flatten` does the same with one attribute.

1.5 A fetch helper

Plan:

- `fetch_all::<T: FromRow>(client, sql, params): query`, then `from_row` on each row, collected into a `Result<Vec<T>, _>`. `fetch_one` and `fetch_opt` beside it.
- Taking tokio-postgres's `GenericClient`, so that a transaction works too (main book Chapter 8).

1.6 Counting the boilerplate

Plan:

- Each field is named three times: in the struct, in the `SELECT` list, and in `try_get`. The struct's name appears in every error message you write by hand.
- What a macro can take over (the `from_row` body, the error context), and what it cannot (the SQL, until Chapter 6).

The programs

- `run`: a hand-written `FromRow` trait and its impl for `Product`, reading a temporary table; asserts the values, including a NULL into an `Option`.
- `run`: by name against by position on 100,000 generated rows; asserts that by position is faster and prints both (the ratio is measured in debug when the chapter is written).
- `run`: the three failures, each matched and its message printed.
- `run`: the nested `Price`, and `fetch_all` over a client and over a transaction.

Side by side

A row into a struct: by hand (this chapter's impl) · shelf (a preview: `#[derive(FromRow)]`, built) · sqlx (`#[derive(sqlx::FromRow)]`, illustration).

The data

A temporary `products` table (`id`, `sku`, `price_cents`, `tags text[]`), and `generate_series` for the measurement.

In stockroom

`TryFrom<&Row> for Store` in stockroom/src/repo/stores.rs, the code this chapter starts from.

Claims

- Measured: by name about 3× by position, release build (PLAN.md §12 #8); the debug ratio (to measure).
- Docs: `Row::try_get` (docs.rs, 0.7.18) and the source of `row.rs`.
- Opinion: once a project has more than a handful of row types, a mapping trait of its own pays for itself.

When not to use a mapping trait

- A single value: `query_one_scalar` (main book Chapter 2).
- A one-off query read in place: `row.get(0)` is clearer than a struct.

Exercises

Plan:

1. Implement `FromRow` for a struct with an `Option<Vec<String>>`. *Hint:* `Option` handles the NULL, `Vec` the array.
2. Make a missing column's error name the struct. *Hint:* map the error, add the name.
3. Measure by name with the columns in the struct's order and in reverse. *Hint:* the search stops at the first match.

> **Open questions.** (1) The ratio in a debug build, which the runnable example sees, and whether a relative assertion stays reliable (to measure). (2) Whether 1.5's `fetch_all` should already prepare first, as shelf's does, or leave that to Chapter 5's story. (3) How much of `row.rs` to quote: the two searches and the `accepts` check, nothing else.

CHAPTER 2

Declarative macros

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part I, third chapter. Builds on Chapter 1 (the hand-written `FromRow`) and on `macro_rules!` as the Rust Reference describes it ("Macros By Example"). shelf: the tuple impls in orm/shelf/src/tuples.rs (built, S1a), from one `macro_rules!` invocation with a line per arity. Crates: tokio-postgres 0.7.18 only. Plan: orm/PLAN.md §7, Chapter 2.

Chapter 1's `FromRow` impl is mechanical: the same line for every field, with the field's name in two places. Mechanical code is what macros are for, and Rust has two kinds. This chapter uses the simpler one, `macro_rules!`, which matches the tokens it is given against patterns and pastes them into a template.

It goes surprisingly far. A `model!` macro can declare a struct and write its `FromRow` impl in one go, and one invocation can write `FromRow` for tuples of every size from one to twelve, which shelf keeps to this day. Then it stops: as soon as a field needs an attribute (read it from another column, skip it, decode it as JSON), the patterns turn into a puzzle, and the errors point at the macro instead of at your code. That is the case for the next chapter's procedural macros.

In this chapter you will learn:

- how `macro_rules!` matches tokens: matchers, fragment specifiers and repetitions
- how to write a `model!` macro that declares a struct and its `FromRow` impl together
- how one macro invocation writes `FromRow` for tuples of one to twelve elements
- what `$crate` is for, and how hygiene keeps a macro's local variables apart from yours
- where declarative macros stop, and why shelf's derives are procedural

---

2.1 Matching tokens

Plan:

- A macro is a list of rules, each a matcher and a transcriber; the first rule that matches wins.
- Fragment specifiers used in the chapter: `ident`, `ty`, `vis`, `tt`, `expr`, `meta`; what each accepts. Repetitions `$( … ),*` and `$( … )+`.
- A **Rust refresher** box: macros expand before type checking, so a macro sees tokens, not types.

2.2 A `model!` macro

Plan:

- `model! { pub struct Product { id: i64, sku: String } }` expanding to the struct and its `FromRow` impl, with `stringify!($field)` as the column name.
- Used against a temporary table, the same assertions as Chapter 1.

- What it costs the user: the struct is now inside a macro call, rust-analyzer sees less, and other derives must be passed through (`$(#[$meta:meta])*`).

2.3 Tuples, one line per arity

Draft prose:

A query that returns two columns does not always deserve a struct. shelf reads it into a tuple, `(String, i64)`, by position: the first element from the first column, and so on. That needs one impl per tuple size, each with a type parameter and a position per element, and a single macro writes all twelve. Each line of the invocation is one arity, with each element's type parameter next to its position: `(A 0, B 1, C 2)`. The position is matched as a `tt`, a single token tree, so that it can be pasted both as a number and, with `stringify!`, as the name of the field in an error.

Plan:

- The excerpt from `orm/shelf/src/tuples.rs`: the macro and the first three lines of its invocation.
- Why not a recursive macro that peels one element at a time: it works, and is Exercise 1, but the flat list is easier to read in an excerpt.

2.4 `$crate` and hygiene

Plan:

- `$crate` in a macro exported from shelf names shelf wherever the macro is used, even if the user renamed the dependency.
- Hygiene: a `let row = …` inside the macro does not capture the caller's `row`; shown with a program where both exist.
- What hygiene does not cover: paths (`Result` means whatever is in scope at the call site), which is why expansions use `::core::result::Result`. Chapter 3 returns to this.

2.5 Where declarative macros stop

Plan:

- Field attributes: `#[col = "sku_code"] sku: String` needs a rule per combination, or a tt-muncher; shown briefly, not built.
- Errors: a mistake inside the invocation is reported against the macro's rules, not as a problem in your struct (the message captured from a real build).
- Generics and where clauses in the matched struct: possible, painful.
- The conclusion: a derive that reads attributes needs to parse Rust, which is what syn does.

The programs

- `run`: the `model!` macro, reading a temporary table.
- `run`: a local trait with tuple impls for one to four elements from one invocation, then shelf's own tuples through `fetch_all::<(String, i64)>` (built).
- `run`: hygiene, a macro's `row` beside the caller's.
- A ~~~`rust` fragment with a broken invocation and its captured error (to capture).

Side by side

Generating impls for many types: by hand (four impls written out) · shelf (the `tuple_impls!` macro, built) · sqlx, which also implements `FromRow` for tuples (check how).

The data

A temporary `products` table, as in Chapter 1.

Claims

- Docs: the Rust Reference, "Macros By Example" (fragment specifiers, hygiene, `$crate`).
- Opinion: use `macro_rules!` for regular repetition; reach for a procedural macro as soon as you need to read attributes or report errors at the user's code.

When not to use `macro_rules!`

- When a generic impl does the job (`impl<T: FromSql> … for T`), with no macro at all.
- When the input needs attributes or good error messages: Chapter 3.

Exercises

Plan:

1. Rewrite the tuple macro recursively, peeling one element per step. *Hint:* the base case is a single element; the recursive rule matches a head and a tail.
2. Make `model!` accept other derives on the struct. *Hint:* `$(#[$meta:meta])*`.
3. Count a struct's fields with a macro. *Hint:* a repetition that expands to `1 +` per field.

> **Open questions.** (1) Whether tt-munching attributes deserves a full example or a paragraph; the chapter's point is where to stop. (2) How sqlx writes its tuple impls (check its source at the version the appendix names). (3) Macro metavariable expressions such as `${index()}` would remove the explicit positions; check whether they are stable in Rust 1.99 (check).

CHAPTER 3

The first derive macro

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part I, fourth chapter. Builds on Chapters 1 and 2. shelf: the proc-macro shell orm/shelf-macros/src/lib.rs, the generator orm/shelf-codegen/src/from_row.rs and the playground's `pretty` helper (all built; the chapter's stage program is S0's simpler generator). Crates: syn 3.0.9, quote 1.0.47, proc-macro2 1.0.107, prettyplease 0.3.0. Decisions: PLAN.md §10 (three crates; syn 3). Findings: §12 #2, #3, #5, #7, #12. Plan: orm/PLAN.md §4.1, §7, Chapter 3.

A procedural macro is a Rust function that runs inside the compiler. `#[derive(FromRow)]` hands it the tokens of your struct; it returns more tokens, an `impl` block, which the compiler compiles as if you had written it. There is no pattern language this time: the function can parse the struct into a syntax tree, look at every field and attribute, and build whatever code it needs.

This chapter writes the first version of shelf's derive: every named field read from the column of its name, nothing more. Most of the work is not the generator, which is twenty lines of syn and quote, but the setting around it: the crate it must live in, the crate it should live in instead, the paths that keep the generated code working in any module, and the ways to see what was generated.

In this chapter you will learn:

- what a proc-macro crate is, and what the compiler hands a derive
- why shelf splits its macros into a code generator and a thin proc-macro shell
- how syn 3 parses a struct, and how quote builds tokens with interpolation and repetition
- what hygiene means for a derive: absolute paths, `__private`, raw identifiers
- three ways to see an expansion: printing it, `cargo expand`, and rust-analyzer
- how a derive reports an error as a value, pointing at the right token

---

3.1 Proc-macro crates

Plan:

- `[lib] proc-macro = true`; the function `#[proc_macro_derive(FromRow)]`, from `proc_macro::TokenStream` to `proc_macro::TokenStream`.
- A **Macro lens** box: the derive receives the whole item, attributes included, and its output is added next to the item, not in its place.
- What a proc-macro crate cannot do: export anything but macros, or run outside the compiler.

3.2 Three crates

Draft prose:

Because a proc-macro crate can export nothing but macros, a test in another crate cannot call the function that generates code, and neither can an example in this book. shelf therefore keeps the generator in `shelf-codegen`, an ordinary library whose functions go from a parsed `syn::DeriveInput` to a `proc_macro2::TokenStream`, and leaves `shelf-macros` a shell of three lines per derive: parse, call the generator, and turn an error into a compile error. `proc_macro2` is what makes the split possible: it has the same types as the compiler's `proc_macro`, but works in any program.

3.3 Parsing with syn 3

Plan:

- `parse_macro_input!(input as DeriveInput)`; `Data::Struct`, `Fields::Named`, each `Field` with its `ident`, `ty` and `attrs`.
- syn's features (`derive`, `full`) and what shelf enables.
- What changed from syn 2, which most material online shows: nothing a derive uses (PLAN.md §12 #5); the changes are in types, expressions and patterns.

3.4 Generating with quote

Plan:

- `quote! { … }` with `#name`, and repetition `#(#reads,)*`; `format_ident!`.
- The stage program: S0's generator written inline in a single-file example, applied to a struct built with `parse_quote!`, printed with `pretty`, and asserted.
- `pretty`, the playground's helper, printed in full as a mirror of `orm/playground/src/lib.rs`.

3.5 Hygiene

Draft prose:

The generated `impl` lands in the user's module, where any name may mean something else. A user may have their own `Result` alias, a constant called `Ok`, a trait called `Default`. So every path the derive writes is absolute: `::core::result::Result`, `::shelf::FromRow`, and tokio-postgres through `::shelf::__private::tokio_postgres`, which also spares the user a direct dependency on the driver. shelf's hygiene test derives next to all three of those names and must compile.

Plan:

- Raw identifiers: a field `r#type` reads the column `type` (`IdentExt::unraw`, §12 #3).
- `#[automatically_derived]`, and what it tells lints.
- The test, quoted from orm/shelf/tests/derive.rs.

3.6 Seeing the expansion

Plan:

- Printing it: shelf-codegen called from an example, `pretty` on the result (the main way, stable Rust).
- `cargo expand`: a separate install that needs a nightly toolchain (§12 #7; check the install before recommending it).
- rust-analyzer's "Expand macro recursively".

3.7 Errors as values

Plan:

- `syn::Error::new_spanned(token, message)` and `into_compile_error`: an enum refused at its `enum` keyword, a tuple struct at its name.
- Spans: tokens the macro writes carry the call site's span; tokens copied from the input keep theirs, so an error about a field's type points at the field (§12 #12).

The programs

- `run`: S0's generator written inline, applied with `parse_quote!`, printed and asserted.
- `run`: the real `#[derive(shelf::FromRow)]` reading a temporary table with `fetch_all`.
- `run`: `shelf_codegen::from_row::expand` and `pretty`, the expansion printed (captured).
- `run`: the error for an enum, printed from the `syn::Error`.
- Mirror: `<!-- mirror: orm/playground/src/lib.rs#pretty -->`.

Side by side

Attaching generated behaviour to a type: by hand (Chapter 1's impl) · shelf (the derive) · sqlx (`#[derive(sqlx::FromRow)]`).

In shelf

Excerpts of `shelf-macros/src/lib.rs` (the shell) and `shelf-codegen/src/from_row.rs` (with the parts later chapters add elided).

Claims

- Docs: syn 3.0.0's release notes (§12 #5); the Rust Reference on procedural macros.
- Opinion: the three-crate split pays for itself with the first test.

When not to write a derive

- A handful of types: write the impls, as Chapter 1 did.
- When a blanket impl over a trait bound does the job.

Exercises

Plan:

1. Generate a `const FIELDS: &[&str]` beside the impl. *Hint:* a second item in the same `quote!`.
2. Make the enum error point at the first variant instead of the keyword. *Hint:* the span of `variants.first()`.
3. Write an expansion test for your derive. *Hint:* compare `pretty` output with a file.

> **Open questions.** (1) Whether to recommend `cargo expand` at all, given the nightly toolchain it needs (§12 #7). (2) How much of the syn 2 → 3 difference to discuss: probably one paragraph and a link. (3) The stage program reproduces S0's generator, which no longer exists in the crate; check that the excerpts of the finished generator still read as its continuation.

CHAPTER 4

Attributes, errors and generics

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part I, fifth chapter. Builds on Chapter 3, and on main book Chapter 3 (`Json<T>`) and Chapter 5 (`FromSql`). shelf: attrs.rs, case.rs, generics.rs, `shelf::FromColumn` and the UI tests in `orm/shelf/tests/ui/` (all built, S1a). Crates: syn 3.0.9, trybuild 1.0.122. Decisions: PLAN.md §10 (attributes parsed by hand, not with darling). Findings: §12 #9, #10, #11, #12. Plan: orm/PLAN.md §4.3, §7, Chapter 4.

A real schema does not match your structs one to one. A column is called `sku_code` but the field is `sku`; a legacy table uses `camelCase`; a field holds a value computed in Rust and is not in the table at all; a `jsonb` column holds a document you want as a struct; a `bigint` should become a `Cents` that refuses negative values. Each of those is one attribute on a field, and serde has taught every Rust developer to expect them.

This chapter teaches the derive to read `#[shelf(…)]`. Parsing is the easy part. The work is in what happens when the attributes are wrong: a misspelt key, a value where none belongs, two attributes that contradict each other. A good derive reports all of them in one build, each at the token that caused it, in words the user understands. The same care goes into generic structs, which need exactly the bounds their fields require, and no others.

In this chapter you will learn:

- how a derive declares a helper attribute, and what happens if it forgets
- how to parse `#[shelf(…)]` with `parse_nested_meta` into a typed struct
- `rename_all`, `rename`, `skip`, `flatten`, `json` and `try_from`: what each generates
- how to report every mistake in one build, each at the right token
- how to test a derive's errors with trybuild, and what reading them taught shelf
- how to give a generic struct the bounds its fields need, and why shelf refuses lifetimes

4.1 Helper attributes

Draft prose:

A derive must say which attributes belong to it: `#[proc_macro_derive(FromRow, attributes(shelf))]`. Without that declaration the compiler does not pass `#[shelf(…)]` to anyone; it rejects it, before the derive runs, with "cannot find attribute `shelf` in this scope", and adds, unhelpfully, that `shelf` is a crate. shelf hit exactly that when its hygiene test first used `#[shelf(skip)]` (PLAN.md §12 #9).

4.2 Parsing with `parse_nested_meta`

Plan:

- `Attribute::parse_nested_meta`, key by key: `meta.path.is_ident("rename")`, `meta.value()?` then a `LitStr`; flags with no value; a type after `=` (`try_from = i64`, or quoted as serde writes it).
- Into a typed `FieldAttrs` with each value's span kept for later errors; duplicates refused.
- Every derive parses every key and ignores the ones it does not use, so `Table`'s keys will not trouble `FromRow` (PLAN.md §4.3).

4.3 The six attributes

Plan:

- For each, the generated line, printed by running the generator: `rename` changes the column name; `rename_all` applies one of four conventions written by hand (`case.rs`); `skip` writes `Default::default()`; `flatten` calls the nested type's `FromRow`; `json` reads through `Json<T>`; `try_from` reads one type and converts.
- One program that uses all six against a table with a `jsonb` column.

4.4 Every error at once

Plan:

- `syn::Error::combine`: shelf collects errors across fields and attributes (`attrs::Errors`) and returns them together.
- Contradictions, each reported at the attribute that does not fit: `skip` with anything, `flatten` with `rename`, `json` with `try_from`.
- Where errors land (§12 #12): at a literal, at a key, at a field's type.

4.5 UI tests

Draft prose:

An error message is part of a derive's interface, so shelf tests it like one. trybuild compiles each file in `tests/ui/`, expects it to fail, and compares the compiler's output with a `.stderr` file beside it. Reading those files for the first time changed shelf: for a field whose type tokio-postgres cannot read, the error named `shelf::__private::tokio_postgres::types::FromSqlOwned`, a path no user should see. shelf now bounds field types by a trait of its own, `FromColumn`, implemented for every type the driver can read and carrying `#[diagnostic::on_unimplemented]`, so the same mistake reads "`Instant` cannot be read from a PostgreSQL column" (§12 #11).

Plan:

- The `.stderr` excerpts: `conflicts.stderr` and `not_readable.stderr`.
- `TRYBUILD=overwrite`, and why the output is pinned to one toolchain (rustc 1.99.1).

4.6 Generics

Plan:

- `split_for_impl`, and the predicates shelf adds only for fields whose type mentions a type parameter (`T: FromColumn`, `M: FromRow`, `Json<Vec<T>>: FromColumn`), from `generics.rs`.
- Why not bound every parameter: a bound on a concrete type would be checked at the derive, far from the field.
- Lifetimes refused with an error: borrowing from a row would make `FromRow` generic over a lifetime and `fetch_all` impossible to write simply; owned values cost an allocation per string, which Chapter 5's numbers put in proportion.

4.7 What darling would save

Plan:

- The same `FieldAttrs` with darling 0.24 (built on syn 3): a struct and a derive; what it handles (unknown keys, duplicates) and what it hides.
- Why shelf parses by hand: this book is about the mechanism.

The programs

- `run`: a stage program that parses attributes with `parse_nested_meta` from a `parse_quote!` struct and prints the result.
- `run`: three mistakes in one struct, the combined errors printed.
- `run`: all six attributes against a temporary table with a `jsonb` column.
- `run`: a generic `Counted<T>` with a flattened `T`.
- Excerpts of `.stderr` files from `orm/shelf/tests/ui/`, checked by `book.py excerpts`.

Side by side

Attribute-driven mapping: by hand (Chapter 1, with the renames written out) · shelf · sqlx (`#[sqlx(rename, flatten, json, try_from)]`, check the exact spelling).

In shelf

`attrs.rs` (the parser and the combination checks), `case.rs`, `generics.rs`, and `FromColumn` in orm/shelf/src/row.rs.

Claims

- Measured: none; the error texts are captured from rustc 1.99.1 (§12 #10, #11).
- Docs: syn's `parse_nested_meta`; the Rust Reference on the `diagnostic` attribute namespace (check the version it was stabilized in).
- Opinion: keep the attribute set small; each attribute is a promise to document and test.

When not to use an attribute

- When the SQL can do it: `SELECT sku_code AS sku` needs no `rename`.
- When the conversion is business logic: read the raw value and convert in a constructor.

Exercises

Plan:

1. Add `#[shelf(default)]`: read the column if present, else `Default`. *Hint:* `resolve` returns an `Option<usize>` for that field.
2. Refuse two fields that map to the same column. *Hint:* collect names in a `HashMap` with their spans.
3. Explain why `FromRow<'a>` would make `fetch_all` hard. *Hint:* who owns the rows?

Open questions. (1) An unreadable field type is reported twice, once per generated use (§12 #11); one bound site would make it once. Decide before writing. (2) Duplicate column detection does not exist yet (Exercise 2, or shelf itself?). (3) The darling comparison needs code, written and compiled against darling 0.24.1 (to write).

CHAPTER 5

Checking a statement once, not every row

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part I, sixth chapter. Builds on Chapters 1 and 4, and on main book Chapter 4 (`prepare`, `Statement::columns`). shelf: `FromRow::{Columns, resolve, from_row_at}`, `from_rows` and `fetch_*` in row.rs and fetch.rs, and the benchmark bench_mapping.rs (all built, S1a). Crates: tokio-postgres 0.7.18. Findings: PLAN.md §12 #4, #8, #13. Plan: orm/PLAN.md §4.0, §7, Chapter 5.

Chapter 1 measured what a lookup by name costs, and Chapter 3's derive does one per field per row. Everything that lookup finds depends only on the statement, not on the row: the position of `sku` is the same in the first row and in the millionth. So shelf finds each field's column once, when it sees the statement, and then reads every row by position.

The same step is the right moment to check types. Instead of discovering on the first row that `price_cents` is `text` and the field is `i64`, shelf checks every field against the statement's columns before the query runs, and reports the problem with the struct's name, the field's and the column's. Because tokio-postgres prepares a statement anyway when you pass it a string, the check costs no extra round trip.

In this chapter you will learn:

- what a statement knows about its columns before any row exists
- how `resolve` finds each field's column and checks its type once per statement
- what the derive generates for this: a `Columns` tuple and `from_row_at`
- why a wrong mapping fails before the query runs, even an `INSERT … RETURNING`
- what it is worth, measured on 100,000 rows

5.1 What a lookup by name costs, again

Draft prose:

On 100,000 rows of twelve columns, reading a struct by name took 38.2–38.5 ms in a release build, and reading it by position 12.4–12.6 ms (PLAN.md §12 #13, Apple Silicon, PostgreSQL 18.6 on localhost). The decoding is the same in both; the difference is the searching. Checking each column's type on every row, which tokio-postgres does whether you ask by name or by position, made no measurable difference (§12 #8).

5.2 A statement knows its columns

Plan:

- `client.prepare(sql)` returns a `Statement` whose `columns()` give each column's name and type, before any row; a `Row` carries the same columns.
- `resolve(columns) -> Result<Columns, Error>`: each field's position, found by exact name (PostgreSQL folds unquoted names itself, so shelf does not retry ignoring case, unlike `Row::try_get`; §12 #4).

5.3 Types checked once

Plan:

- `FromSql::accepts(column.type_())`, called once per field per statement, under shelf's `FromColumn` bound.
- The errors: `Mapping::MissingColumn`, `Mapping::WrongType` (with the SQL type and the Rust type, shortened: `Option<String>`, not `core::option::Option<alloc::string::String>`), and, per row, `Mapping::Null` and `Mapping::Decode`.
- The proof that the check comes first: an `INSERT … RETURNING` whose result lacks a field's column fails, and the table still has the rows it had (from `tests/from_row.rs`).

5.4 What the derive generates

Plan:

- The expansion of a two-field struct, printed: `type Columns = (usize, usize)`, `resolve` calling `__private::column::<T>` per field, `from_row_at` calling `__private::get::<T>` with `columns.0`, `columns.1`.
- `flatten` puts the nested type's `Columns` inside the tuple; `skip` puts `()`.
- `from_row` stays, as a provided method that resolves and reads one row.

5.5 Measured

Draft prose:

shelf resolving once took 13.0–13.1 ms on the same 100,000 rows: about 4% over the hand-written code that reads by position, and a third of the time of reading by name. Resolving again for every row, which is what calling `from_row` in a loop does, took 38.2–38.4 ms, as much as the lookups by name it repeats (§12 #13). That is why `fetch_all` and `from_rows` resolve once, and why the chapter's advice is to use them.

Plan:

- The table printed by `cargo run -p shelf --release --example bench_mapping`, pasted as captured.

5.6 Where it is used

Plan:

- `fetch_all`, `fetch_one`, `fetch_opt`: prepare, resolve against the statement, query.
- `from_rows` for rows you already have; streams resolve once too (Chapter 12).

The programs

- `run`: `fetch_all` into a struct, and the same rows through `from_row` in a loop, with both times printed and a relative assertion (debug build; to measure).
- `run`: the `INSERT … RETURNING` that never runs, and the row count unchanged.
- `run`: a wrong type reported for a query that returns no rows.
- The benchmark, as a `bash` command and its table (release; not run by `book.py check`).

Side by side

Reading 100,000 rows: by hand, by position · shelf (`from_rows`) · sqlx, whose rows look up names in a map built per statement (check in sqlx-postgres's source).

In shelf

The trait from `row.rs`, `column` and `get` from `row.rs`, and `fetch_all` from `fetch.rs`.

Claims

- Measured: §12 #8 and #13 (release, Apple Silicon, PG 18.6); the debug ratio (to measure).
- Docs: `Statement::columns`, `Column::type_` (docs.rs, 0.7.18).
- Opinion: an error before the first row is worth more than the speed.

When not to resolve once

- A single row: `from_row` is the same work, and simpler to call.
- Rows from different statements: their `Columns` are not interchangeable.

Exercises

Plan:

1. Prepare a statement once, resolve once, and run it twice with different parameters. *Hint:* `Columns` belongs to the statement, not to the execution.
2. Measure with three columns instead of twelve. *Hint:* the search grows with both.
3. Make `resolve` refuse a result with two columns of the same name. *Hint:* a join's `id`.

> **Open questions.** (1) Whether a relative timing assertion is reliable in a debug build (to measure). (2) How sqlx finds columns by name (check). (3) Whether S1b's executor should cache `Columns` with each prepared statement, so that `resolve` runs once per connection rather than once per call; a design question for Chapter 7.

CHAPTER 6

Tables: the model derive

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part II, first chapter. Builds on Chapters 3–5, and on main book Chapter 2 (`quote_ident`), Chapter 4 (the "cached plan must not change result type" error) and Chapter 14 (the repository layer). shelf: `#[derive(Table)]` in `shelf-codegen/src/table.rs`, the `Table` trait and `Error::NotFound` (proposed, S1b step 3). Crates: syn 3.0.9, tokio-postgres 0.7.18. Decisions: PLAN.md §10 (SQL as literals generated at compile time). Plan: orm/PLAN.md §4.2, §4.3, §7, Chapter 6; §13, S1b step 3.

`FromRow` reads any query's rows, but you still write every query. Most of them are the same few: fetch one row by its key, fetch them all, count them. The struct already knows its columns; what it lacks is the table they come from and which of them is the key. Two attributes supply that, and with them the derive can write the everyday `SELECT`s itself.

It writes them at compile time. The macro knows every name, so it can assemble each statement as a string literal, with every identifier quoted, and leave nothing to build at run time. Values never go into those strings: they stay `$1` parameters, as the main book insisted from its second chapter. What the macro cannot put in a literal, because it lives in another type, is where `LazyLock` comes in.

In this chapter you will learn:

- what a table adds to a row type: a name, a key and a column list
- how `#[derive(Table)]` reads `table`, `schema`, `primary_key` and `generated`
- how a derive writes SQL at compile time, quoting identifiers so that a table called `order` works
- `find`, `find_opt`, `all` and `count`, and the SQL each one sends
- why shelf never writes `SELECT *`
- where const evaluation stops, and what `LazyLock` is for

6.1 What a table adds

Plan:

- A model is a `FromRow` struct plus a table: `TABLE`, `COLUMNS`, the key's type and a way to get it from a value (proposed names, settled in S1b).
- `Table` requires `FromRow` (a supertrait): a model is always readable.

6.2 `#[derive(Table)]`

Plan:

- `#[shelf(table = "products")]` on the struct (and `schema` when needed); `#[shelf(primary_key)]` on one field, or on several for a composite key read as a tuple; `#[shelf(generated)]` for columns the database fills (`uuidv7()`, `now()`, identity), which writes will leave out (Chapter 8).
- The new keys added to `attrs.rs`; `FromRow` ignores them, so the UI test that refuses `table` today changes (PLAN.md §13).

6.3 SQL at compile time

Plan:

- The derive builds `SELECT "id", "sku", … FROM "products" WHERE "id" = $1` as a literal and emits it as an associated `const` (proposed).
- Quoting at macro time: a column called `type` or a table called `order` works; the `quote_ident` of the main book's Chapter 2, run by the compiler.
- A **Pitfall** box: an identifier is quoted, a value is a parameter; nothing a user types at run time ever reaches the generated SQL.

6.4 Reading

Plan:

- `find(&db, &key)` (one row, else `Error::NotFound`), `find_opt`, `all`, `count`: each with its SQL printed and its round trips counted.
- An explicit column list, never `*`: a cached statement that selected `*` fails with 0A000 after `ALTER TABLE … ADD COLUMN` (root PLAN.md §12 #42), and `*` would hand `resolve` columns the struct does not have.

6.5 Metadata as consts, and where they stop

Plan:

- `TABLE` and `COLUMNS` as consts are free. A literal cannot include another type's const, though: `concat!` takes literals only, so SQL that needs a `Table`'s name from elsewhere (an `Insert` struct naming its model, Chapter 8) is built once in a `LazyLock<String>`.
- A **Rust refresher** on `LazyLock` (stable since Rust 1.80; check) and why it is not a `const`.

The programs

- `run` (proposed): a model's `TABLE` and `COLUMNS` printed; the generated SQL printed.
- `run`: `find`, `find_opt`, `all` and `count` against a temporary table.
- `run`: a table called `order` with a column called `type`.
- `run`: `find` on a missing key, matching `Error::NotFound`.
- `run`: the expansion of a small model, printed by running the generator.

Side by side

Fetching by primary key: by hand (`fetch_one` with the SQL written out) · shelf (`Product::find(&db, &id)`, proposed) · SeaORM (`Entity::find_by_id(id).one(&db)`, check).

The data

A temporary `products` table with an identity key; a second table with a composite key (`stock (store_id, product_id)`, as in stockroom).

Claims

- Docs: PostgreSQL's rules for identifiers and key words (the documentation's "Lexical Structure"); root PLAN.md §12 #42 for the cached-plan error.
- Opinion: an ORM should make the SQL it sends easy to predict; constants make it easy to print.

When not to use `Table`

- Reports, joins and aggregates: SQL and `fetch_all` with a `FromRow` struct.
- Views: a `FromRow` struct suffices, since there is no key to find by (or declare one; check whether that is useful).

Exercises

Plan:

1. Add `exists(&db, &key)`. *Hint:* `SELECT EXISTS (SELECT 1 FROM … WHERE …)`.
2. Support `schema` and print the qualified name. *Hint:* quote each part separately.
3. Find by a composite key. *Hint:* the key type is a tuple.

> **Open questions.** (1) The trait's exact items and names (`TABLE`, `COLUMNS`, `Key`, `key()`) are proposed; settle them in S1b. (2) How `find` takes a composite key: a tuple of references or of owned values. (3) `concat!` against `LazyLock` (S1b probe). (4) Whether `count` belongs here or in the query builder (Chapter 10).

CHAPTER 7

The executor and shelf's errors

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part II, second chapter. Builds on Chapter 6, and on main book Chapter 4 (round trips), Chapter 6 (`DbError`, `SqlState`), Chapter 8 (the sealed `GenericClient`, the retry helper) and Chapter 13 (deadpool's `prepare_cached`). shelf: the `Executor` trait in `shelf/src/executor.rs`, the `deadpool` feature, `Error::NotFound` and `Error::Stale` (proposed, S1b step 2); `fetch_*` move onto it. Crates: tokio-postgres 0.7.18, deadpool-postgres 0.14.2. Findings: root PLAN.md §12 #6, #33, #42. Plan: orm/PLAN.md §4.5, §7, Chapter 7; §13, S1b steps 1–2.

So far shelf's functions take tokio-postgres's `GenericClient`, which covers a `Client` and a `Transaction`. A service holds neither: it holds a deadpool connection, whose own `GenericClient` adds the one thing a model layer wants most, a statement cache. Both traits are sealed, so shelf cannot implement either for anything, and code generic over one does not accept the other.

The way out is a trait of shelf's own, with only the calls shelf needs, implemented for the four types a program actually has. Deciding what goes in it means deciding how each statement reaches the server, which is a question of round trips the main book taught you to measure. The chapter measures three strategies through lagproxy, picks one per executor, and gives shelf an error type that says what an application can do about each failure.

In this chapter you will learn:

- why neither `GenericClient` will do, and what a trait of shelf's own must contain
- how to write async methods in a trait whose futures are `Send`, and why that matters in axum
- three ways to send a statement (`prepare`, `prepare_cached`, `query_typed`) and their round trips, measured
- how shelf calls run inside a transaction, and with the main book's retry helper
- `shelf::Error`, and how an application maps it to HTTP statuses

7.1 Why not `GenericClient`

Plan:

- tokio-postgres's `GenericClient` is sealed; deadpool's is sealed too, and only it has `prepare_cached` (root CLAUDE.md, "Verified API facts").
- An attempt to implement one for a wrapper, and the captured error.

7.2 The `Executor` trait

Plan:

- The calls shelf needs: prepare (cached where possible), `query`, `query_one`, `query_opt`, `execute` (proposed list).
- Methods returning `impl Future<Output = …> + Send`, and a **Rust refresher** on why the `Send` must be written: a handler's future must be `Send`, and a trait method's future is not known to be unless the trait says so.
- Impls for `tokio_postgres::Client`, `tokio_postgres::Transaction<'_>` and, behind the default feature `deadpool`, `deadpool_postgres::Client` and its `Transaction<'_>`.
- `fetch_*` moved onto it; the tests unchanged.

7.3 Three ways to send a statement

Plan:

- `prepare` then execute: two round trips every time (Parse/Describe, then Bind/Execute).
- `prepare_cached` (deadpool): two the first time, one after; the cache is per connection (root PLAN.md §12 #42: about 1 ms the first time, about 3 µs after).
- `query_typed`: one round trip and no named statement, but the parameter types must be known, and `resolve` has no `Statement` to check against before the query.
- Measured through lagproxy at 5 ms each way (root PLAN.md §12 #33: a round trip cost 15.3 ms); the choice per executor recorded in PLAN.md §10.

7.4 Transactions

Plan:

- shelf calls on a `&Transaction` see the transaction's own writes.
- The main book's `in_transaction` retry helper with an `AsyncFnMut` closure that calls shelf (root PLAN.md §12 #6: the closure must own its data).

7.5 `shelf::Error`

Plan:

- The variants: `Mapping` and `Db` (built), `NotFound` and `Stale` (proposed), each with what the caller can do about it.
- Constraint names stay the application's: stockroom's `AppError` maps `SqlState` and constraint names to statuses; an `impl From<shelf::Error> for AppError` shows the pattern.

The programs

- A `~~~rust` fragment that tries to implement `GenericClient`, with its captured error.
- `run` (proposed): one repository function called with a client, a transaction and a pooled connection.
- `needs=lagproxy`: the three strategies timed; asserts that a cached statement beats prepare-every-time by at least a round trip (relative).
- `run`: the retry helper running shelf calls under `SERIALIZABLE`.
- `run`: a unique violation from shelf mapped to an application error.

Side by side

One repository function over a client, a transaction and a pool: by hand (generic over deadpool's `GenericClient`, as stockroom does) · shelf (`&impl Executor`, proposed) · sqlx (`impl Executor<'_, Database = Postgres>`, check).

Claims

- Measured: root PLAN.md §12 #33 (round trip through lagproxy), #42 (`prepare_cached` timings); shelf's three strategies (to measure).
- Docs: deadpool-postgres 0.14.2's `GenericClient` and `prepare_cached`.
- Opinion: an ORM's executor trait should be small enough to implement for a new client type in ten lines.

When not to use the executor

- Pipelining many independent queries on one connection (main book Chapter 7): use the `Client` directly, or `try_join!` shelf calls on one shared client (check that the executor's futures allow it).

Exercises

Plan:

1. Implement `Executor` for a wrapper that logs each statement's name and duration. *Hint:* delegate, and time around the await.
2. Add a statement cache for a plain `Client`. *Hint:* a wrapper that owns the client and a map.
3. Map `Error::Stale` to 412 and `NotFound` to 404 in an axum handler. *Hint:* main book Chapter 15.

> **Open questions.** (1) The `Send` futures inside axum handlers and the retry helper must be probed before the trait is fixed (S1b step 1). (2) What a plain `Client` does without a cache: prepare every time, or `query_typed`. (3) Where `resolve`'s result lives when statements are cached (Chapter 5's open question).

CHAPTER 8

Writing rows: insert, update, delete, upsert

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part II, third chapter. Builds on Chapters 6 and 7, and on main book Chapter 2 (`RETURNING`, `ON CONFLICT`), Chapter 6 (constraint errors) and Chapter 10 (`MERGE`). shelf: `#[derive(Insert)]` in `shelf-codegen/src/insert.rs`, `update`, `delete`, `delete_by_key` and upsert in `shelf/src/write.rs` (proposed, S1b step 4). Crates: tokio-postgres 0.7.18. PostgreSQL: `uuidv7()` (PG 18), `MERGE … RETURNING` (PG 17). Plan: orm/PLAN.md §4.2–§4.4, §7, Chapter 8; §13, S1b step 4.

Reading is half of a model layer. The other half is writing, and the first question is what type you write. A `Product` read from the database has an `id` and a `created_at`, but a product you are about to create has neither: the database will assign them. Making those fields `Option`s would spread `unwrap()`s through every reader of the model. shelf does what stockroom already does by hand, and gives creation its own struct, `NewProduct`, with only the columns you supply.

`#[derive(Insert)]` on that struct writes the `INSERT`, and `RETURNING` hands back the full model, defaults filled in, in the same round trip. Updates and deletes go through the model itself, by its key, and an upsert is one more clause. Each method's SQL is printed, and each one says what happens when a constraint refuses it.

In this chapter you will learn:

- why creation gets its own struct, and how `#[derive(Insert)]` names its model
- how `RETURNING` gives back the whole model, with the database's defaults, in one round trip
- `update`, `delete` and `delete_by_key`, and what zero rows affected means
- upserts with `ON CONFLICT`, and when `MERGE` is the better statement
- what constraint violations from shelf's writes look like, and where to map them

8.1 `#[derive(Insert)]`

Plan:

- `#[shelf(model = Product)] struct NewProduct { sku, name, price_cents, tags }`: the table and the `RETURNING` list come from the model.
- The SQL: `INSERT INTO "products" ("sku", …) VALUES ($1, …) RETURNING "id", …`, built once (`LazyLock`, Chapter 6) because the model's names are not literals here.
- `new_product.insert(&db) -> Result<Product, Error>` (proposed).

8.2 Database defaults

Plan:

- `uuidv7()` (PG 18; verify on the server), `now()`, identity columns: the model marks them `generated`, the `New…` struct leaves them out.
- What happens if a `New…` struct names a `generated` column: a compile error from the derive (proposed), or the database's.

8.3 Update and delete

Plan:

- `product.update(&db)`: every column that is neither key nor generated, `WHERE` the key. Zero rows affected means the row is gone: `Error::NotFound`.
- `product.delete(&db)`, `Product::delete_by_key(&db, &key)`.
- A **SQL lens** box on what `UPDATE` of every column costs (a new row version either way).

8.4 Upsert

Plan:

- `ON CONFLICT ("sku") DO UPDATE SET … = excluded.…`: the conflict target named by an attribute or an argument (open question).
- `DO NOTHING` returns no row on conflict, so the method returns an `Option`.
- `MERGE … RETURNING merge_action()` (PG 17) as the statement for a batch with several outcomes; main book Chapter 10's import does it by hand.

8.5 Constraint errors

Plan:

- A duplicate `sku`: `Error::Db` carrying 23505 and the constraint's name; the application's mapping (Chapter 7) turns it into 409.
- A `CHECK` violation: 23514, 422.

The programs

- `run` (proposed): insert and read back, with the defaults filled.
- `run`: `uuidv7()` keys ordered by creation time.
- `run`: update, then delete, then update again (`NotFound`).
- `run`: an upsert twice, the second updating.
- `run`: a duplicate key's error, its constraint name printed.

Side by side

Inserting a row and reading it back: by hand (stockroom's `stores::create`) · shelf (`NewStore { … }.insert(&db)`, proposed) · SeaORM (`ActiveModel { … }.insert(&db)`, check).

In stockroom

`create` in stockroom/src/repo/stores.rs: the `New…` struct and the `INSERT … RETURNING` this chapter generates.

The data

A temporary `products` table with an identity key and a unique `sku`; a `uuid` key with `DEFAULT uuidv7()` for 8.2.

Claims

- Docs: PostgreSQL's `INSERT … ON CONFLICT`, `MERGE` (`RETURNING` since PG 17) and `uuidv7()` (PG 18); verify each on the 18.6 server before writing.
- Opinion: a separate struct for creation is clearer than `Option` fields on the model.

When not to use shelf's writes

- A guarded update such as stockroom's oversell check (`UPDATE … WHERE available >= $3`): the condition is the point, so write it.
- Bulk loads: Chapter 12.

Exercises

Plan:

1. Add `insert_returning_id` that returns only the key. *Hint:* a second `RETURNING` list.
2. Make `update` refuse a model whose key is a default value. *Hint:* there is no row to update.
3. Write the upsert with `MERGE` and compare. *Hint:* `merge_action()` tells you which branch ran.

> **Open questions.** (1) Naming the model (`model = Product`) and building the SQL with `LazyLock` (S1b probe). (2) How the upsert's conflict target is given: an attribute on the `New…` struct, or an argument. (3) Whether `update` writes every column or only changed ones (that is `Patch`, Chapter 9).

CHAPTER 9

Partial updates and optimistic locking

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part II, fourth chapter. Builds on Chapter 8, and on main book Chapter 4 (statements per shape), Chapter 8 (§8.8, optimistic concurrency) and Chapter 15 (ETag and `If-Match`). shelf: `#[derive(Patch)]` in `shelf-codegen/src/patch.rs`, the `version` and `updated_at` attributes, `Error::Stale` (proposed, S1b step 5). Crates: tokio-postgres 0.7.18, serde. Findings: root PLAN.md §12 #1 (domain-typed parameters). Plan: orm/PLAN.md §7, Chapter 9; §13, S1b steps 1 and 5.

An admin edits a product's price and nothing else. The request body is `{"price_cents": 990}`, and the update should touch that column only; another admin's change to the name, made a second earlier, must survive. Chapter 8's `update` writes every column, so it would undo that change. A partial update needs to know which fields were sent, and JSON has a third state that `Option` cannot express: a field that was sent as `null`, which means "set it to NULL", is not a field that was left out.

Then there is the edit that arrives late. Two tabs load version 7 of the product; both save. Without a check, the second save silently overwrites the first. With a version column, the second `UPDATE` finds no row with version 7, and the application can say so. This chapter builds both: `#[derive(Patch)]` in one of three designs, chosen by measurement, and a `version` attribute that turns a lost update into an error.

In this chapter you will learn:

- how to tell "absent" from "null" in a request body, with `Option<Option<T>>` and serde
- three ways to write a partial `UPDATE`, and what each costs in statements and round trips
- how `#[derive(Patch)]` generates the one shelf chose
- how a version column turns a lost update into `Error::Stale`, and how to tell it from `NotFound`
- what `updated_at = now()` records (the transaction's start, not the statement's)
- how ETags and `If-Match` fit on top, in axum

9.1 Absent or null

Plan:

- `Option<T>` for a non-nullable column (absent or a value), `Option<Option<T>>` for a nullable one (absent, null, or a value).
- serde: deserializing the double `Option` needs a helper (a `deserialize_with` or a small crate; check what is current).

9.2 Three designs

Draft prose:

There are three ways to send a partial update, and each gives something up. The first builds the `SET` list at run time from the fields that are present: the SQL is exactly what you would write by hand, but every combination of fields is a different statement, prepared and cached separately. The second sends every column, and writes `col = coalesce($n, col)`: one statement for every patch, but `coalesce` cannot tell a NULL you want from a value you did not send, so it can never set a column to NULL. The third sends a flag beside each value, `col = CASE WHEN $f THEN $v ELSE col END`: one statement, NULLs included, at the price of twice the parameters.

Plan:

- Each design run against the same table; prepared statements counted in `pg_prepared_statements`, round trips through lagproxy (to measure).
- The choice recorded in PLAN.md §10.
- The catch for design (c): an unset field still sends a typed value; check that a `None` of the field's type serializes for every column type, domains included (root PLAN.md §12 #1).

9.3 Version columns

Plan:

- `#[shelf(version)]` on an `i32` field: `WHERE "id" = $1 AND "version" = $2`, and `"version" = "version" + 1` in the `SET` list.
- Zero rows: either the row is gone (`NotFound`) or the version moved on (`Stale`). Telling them apart in one round trip with a CTE that also reports whether the row exists (S1b probe).
- The main book's §8.8 did this by hand; stockroom's `PATCH /products/{sku}` does it with `coalesce($2, version)`.

9.4 `updated_at`

Plan:

- `#[shelf(updated_at)]` writes `"updated_at" = now()`.
- A **SQL lens** box: `now()` is the transaction's start time; two updates in one transaction record the same instant (`clock_timestamp()` does not).

9.5 ETags in axum

Plan:

- The version as the ETag; `If-Match` into the patch's expected version; `Stale` ⟶ 412 with the current version, as stockroom does.
- A handler driven with `oneshot`.

The programs

- `run` (proposed): absent, null and a value deserialized into a patch struct.
- `run`: the three designs against one table, with the prepared statements counted.
- `run`: two tabs on one row, the second save getting `Error::Stale`.
- `run`: `updated_at` twice in one transaction, the same instant.
- `run`: an axum handler with `If-Match`, through `oneshot` (412 on a stale version).

Side by side

An edit from two admin tabs: by hand (stockroom's `PATCH /products/{sku}`) · shelf (`ProductPatch { … }.apply(&db, &id, version)`, proposed) · SeaORM (`ActiveModel` with `Set` and `NotSet`, check).

In stockroom

The `update` in stockroom/src/repo/products.rs: `coalesce` for the fields and for the version.

Claims

- Measured (to measure): statements prepared per design, round trips per patch.
- Docs: PostgreSQL's date/time functions (`now()` against `clock_timestamp()`).
- Opinion: design (c) is the right default for a model layer; (a) when the SQL must look hand-written.

When not to use `Patch`

- A change that is a business operation (an order's status transition, stockroom's `POST /orders/{id}/transition`): write a function with its own rules.
- Counters (`stock = stock - 1`): an expression, not a value from the client.

Exercises

Plan:

1. Make a patch with no fields a no-op that sends nothing. *Hint:* check before the round trip.
2. Return the changed columns' names along with the model. *Hint:* the flags already say which.
3. Add `#[shelf(version)]` on a `bigint`. *Hint:* the bound on the field's type.

> **Open questions.** (1) Which design, by measurement (S1b step 5). (2) Telling `Stale` from `NotFound` in one round trip (S1b probe). (3) The serde helper for `Option<Option<T>>` (check which crate or function is current).

CHAPTER 10

A typed query builder

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, first chapter. Builds on Chapters 6 and 7, and on main book Chapter 2 (parameters, injection), Chapter 3 (`= ANY($1)` with arrays) and Chapter 9 (keyset pagination). shelf: `Column<M, T>` constants generated by `Table`, and the query module `shelf/src/query/` (proposed, S1c). Crates: tokio-postgres 0.7.18. Plan: orm/PLAN.md §4.4, §7, Chapter 10.

`find` and `all` cover two queries. The third most common is "these rows, filtered, sorted, a page at a time", and its filter changes with the request: a tag if the client sent one, a price range if it sent that, both or neither. Writing that SQL by hand means assembling a string and a parameter list in step, numbering each `$n` correctly, which is exactly the code where injection bugs and off-by-one parameter errors live.

A query builder assembles it for you, from typed pieces. The derive already knows every column and its Rust type, so it can generate a constant per column, `Product::PRICE_CENTS`, that knows both. A filter built from it, `Product::PRICE_CENTS.gt(1_000)`, can only take an `i64`, and a mistake is a compile error at the call. The builder numbers the parameters, keeps the values out of the SQL, and stays small enough to read in one sitting.

In this chapter you will learn:

- what a query builder should type (columns and values) and what it should leave to SQL
- how `Table` generates a `Column<M, T>` constant per field, and what `PhantomData` does there
- how expressions become SQL with numbered parameters and an owned parameter list
- ordering, limits and keyset pages
- which mistakes fail to compile, shown with UI tests
- the escape hatch: raw SQL, inside a filter or instead of the builder

---

10.1 What to type

Plan:

- Typed: the column (it belongs to this model) and the value (it has the column's type).
- Left to SQL: functions, joins, subqueries; the builder accepts a raw fragment with its parameters instead of modelling them.

10.2 Column constants

Plan:

- `Product::PRICE_CENTS: Column<Product, i64>` (proposed names), generated from each field; the quoted column name inside.

- A **Rust refresher** on `PhantomData<(M, T)>`: a zero-sized marker that makes the type carry the model and the value type.

10.3 Expressions

Plan:

- `eq`, `ne`, `lt`, `le`, `gt`, `ge`, `between`, `like`, `is_null`, `any` (`= ANY($n)` with a `Vec`), `contains` (`@>` for arrays); `and`, `or`, `not`.
- An expression tree rendered once into SQL text and a parameter list, numbering `$1…$n` as it goes; the rendered SQL printed in the chapter.

10.4 Owned parameters

Plan:

- The query owns its values (`Box<dyn ToSql + Sync + Send>`, proposed), so it can be built in one function and run in another, across an await.
- The cost: one allocation per parameter; nothing compared with a round trip.

10.5 Order, limit and pages

Plan:

- `order_by(Product::SKU.asc())`, `limit`, `offset`; `count` and `exists` on the same filter.
- Keyset pagination with `after(Product::SKU, last)` (main book Chapter 9's recommendation), and why `offset` gets slower page by page.

10.6 What fails to compile

Plan:

- A wrong value type (`Product::PRICE_CENTS.eq("x")`), a column of another model in a `Product` query: UI tests and their captured errors.
- What still fails at run time: a raw fragment, and a column whose type changed in the database (Chapter 13).

10.7 The escape hatch

Plan:

- `fetch_all::<T>` with hand-written SQL, as always.
- A raw fragment inside a filter, with its own parameters renumbered by the builder.

The programs

- `run` (proposed): a filtered, sorted page against a temporary table.
- `run`: the rendered SQL and its parameters printed for three filters.
- `run`: keyset pages until the end.
- UI tests excerpted: the wrong value type, the wrong model.
- `run`: a raw fragment inside a filter.

Side by side

A filtered, sorted page: by hand (stockroom's keyset `page`) · shelf (`Product::query().filter(…).order_by(…).limit(20).all(&db)`, proposed) · diesel (`products.filter(price_cents.gt(1000)).order(sku.asc()).limit(20)`, check).

In stockroom

`page` in stockroom/src/repo/products.rs: the optional tag filter written as `($2::text IS NULL OR tags @> ARRAY[$2])`, which a builder replaces with a filter that is simply absent.

Claims

- Opinion: a query builder earns its place for optional filters; for fixed queries, plain SQL is clearer.
- Docs: PostgreSQL's array operators (`@>`, `= ANY`).

When not to use the builder

- Reports, aggregates, CTEs, window functions: SQL with a `FromRow` struct.
- A fixed query run often: a hand-written statement caches as one statement; each filter shape is a different statement (the open question below).

Exercises

Plan:

1. Add `ilike`. *Hint:* one more operator in the renderer.
2. Add `in_range` for `Option` bounds, skipping a missing bound. *Hint:* build the expression conditionally.
3. Render a query and run it through `fetch_all` yourself. *Hint:* the builder's output is SQL and parameters.

> **Open questions.** (1) Ergonomics: `&str` for a `String` column and an `i32` literal for an `i64` column; probe the trait design before writing (PLAN.md §11). (2) Statement caching with dynamic SQL: each filter shape becomes a statement. (3) Whether `count` lives here or on `Table` (Chapter 6).

CHAPTER 11

Relations without N+1

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, second chapter. Builds on Chapters 6, 7 and 10, and on main book Chapter 3 (arrays as parameters), Chapter 7 (pipelining) and lagproxy (apps/lagproxy). shelf: the `belongs_to` and `has_many` attributes, their generated methods, and `shelf/src/relations.rs` (proposed, S1c). Crates: tokio-postgres 0.7.18. Findings: root PLAN.md §12 #21, #33. Plan: orm/PLAN.md §4.3, §7, Chapter 11.

An order has lines. Showing a page of fifty orders with their lines is the textbook case where an ORM is either a great help or a disaster. The disaster is the obvious loop: fetch the orders, then, for each one, fetch its lines. That is fifty-one queries, and over a network each one costs a round trip, so the page that took a millisecond on localhost takes a second in production. It is called the N+1 problem, and the cure is older than ORMs: fetch all the lines for all fifty orders in one query, and sort them into their orders in Rust.

shelf makes the cure the easy path. A `has_many` attribute on `Order` generates a loader that takes a slice of orders and returns their lines grouped by order, in one query with `order_id = ANY($1)`. The chapter measures the loop and the loader through lagproxy, then looks at the two other answers, a join and pipelining, and explains why shelf has no lazy loading at all.

In this chapter you will learn:

- how `belongs_to` and `has_many` declare a relation, and the methods they generate
- what the N+1 problem costs, measured through lagproxy
- how batched loading with `= ANY($1)` fetches every child in one round trip
- how to pipeline independent loads on one connection
- when a join is the better query, and what it costs in duplicated rows
- why shelf has no lazy loading

11.1 Declaring relations

Plan:

- `#[shelf(has_many(OrderLine, key = order_id, as = lines))]` on `Order`; `#[shelf(belongs_to(Order, key = order_id))]` on `OrderLine` (proposed syntax; PLAN.md §4.3). Names are explicit: no pluralizing.
- Generated (proposed): `order.lines(&db)`, `Order::load_lines(&db, &orders)`, `line.order(&db)`.

11.2 N+1, measured

Draft prose:

Through lagproxy at 5 ms each way, one round trip cost 15.3 ms on this machine (root PLAN.md §12 #33). Fifty orders and their lines fetched one order at a time is fifty-one of them, three quarters of a second before

the server has done any real work, and the time grows with the page size. The loop is not slow because PostgreSQL is slow; it is slow because it waits for the network fifty-one times.

Plan:

- The loop and its time through lagproxy (to measure); the server's own time from `EXPLAIN ANALYZE` beside it, to show where the time goes.

11.3 Batched loading

Plan:

- One query, `WHERE order_id = ANY($1)` with the orders' keys as an array, then a `HashMap` from key to children; the loader returns the children in the parents' order.
- Measured through lagproxy: two round trips for the page, whatever its size (to measure).
- The foreign key type needs `Hash + Eq`; a composite key needs a tuple (open question).

11.4 Pipelining independent loads

Plan:

- Lines and the store for a page of orders: two loaders, independent, joined with `try_join!` on one connection, costing about one round trip (main book Chapter 7; §12 #21: one pipelined client beat a pool over latency).
- Which executors allow it: a shared `Client`, not a `Transaction` held mutably (check with the executor of Chapter 7).

11.5 Joins

Plan:

- A join returns each order once per line: duplicated parent columns, and a `FromRow` with `flatten` to read them.
- `json_agg` or `array_agg` of a composite to return one row per order; when that wins (a small number of children, one round trip).

11.6 Why no lazy loading

Plan:

- Lazy loading needs a hidden connection in the model and a hidden round trip in a field access; in async Rust it would also need `.await` on a getter. shelf makes every round trip visible instead (opinion, argued).

The programs

- `needs=lagproxy` (proposed): the N+1 loop timed.
- `needs=lagproxy`: the batched loader timed; asserts it is at least ten times faster than the loop for fifty orders (relative; to measure).
- `needs=lagproxy`: two loaders pipelined with `try_join!`.
- `run`: a join with `flatten`, and `json_agg` into `Json<Vec<…>>`.

Side by side

Orders with their lines: by hand (stockroom's `orders::get`, two pipelined queries) · shelf (`Order::load_lines`, proposed) · SeaORM (`find_with_related`, check).

The data

Temporary `orders` and `order_lines` tables with fifty orders and a few lines each; stockroom's schema in the capstone.

Claims

- Measured: root PLAN.md §12 #33 (a round trip through lagproxy), #21 (pipelining against a pool); N+1 against batched (to measure).
- Opinion: no lazy loading; every round trip should be visible at the call site.

When not to use relation loaders

- A report across several tables: one SQL query.
- A deep tree (categories of categories): a recursive CTE.

Exercises

Plan:

1. Load lines and products for a page of orders, two levels. *Hint:* the second loader takes the first one's children.
2. Return lines ordered by a column, inside each group. *Hint:* order in SQL, group in Rust.
3. Write the `json_agg` version as a `FromRow` struct. *Hint:* `#[shelf(json)]`.

> **Open questions.** (1) The names and signatures of the generated methods, and whether the loader returns a `HashMap` or a `Vec` of pairs. (2) Composite foreign keys. (3) Whether loaders can be pipelined through the `Executor` trait as designed in Chapter 7 (check).

CHAPTER 12

Bulk writes and streams

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, third chapter. Builds on Chapters 5, 7 and 8, and on main book Chapter 5 (domains and arrays), Chapter 9 (`RowStream`, backpressure, `pin!`) and Chapter 10 (COPY, binary COPY, pgload). shelf: `insert_many` (`unnest`), a binary COPY writer generated by the derive, and streams of models, in `shelf/src/bulk.rs` and `shelf/src/stream.rs` (proposed, S1c). Crates: tokio-postgres 0.7.18, futures-util. Findings: root PLAN.md §12 #1, #4, #22, #34–#37. Plan: orm/PLAN.md §7, Chapter 12.

Inserting ten thousand rows one `insert` at a time is the N+1 problem again, in the other direction: ten thousand round trips. The main book measured it with pgload: one `INSERT` per row was about a hundred times slower than binary COPY on localhost, and pipelining or `unnest` batches recovered most of the difference (root PLAN.md §12 #22). A model layer that offers only `insert` pushes every bulk load into that slow path.

shelf adds two bulk writes, both generated from the `New…` struct the reader already has: `insert_many`, one `INSERT … SELECT * FROM unnest(…)` with an array per column, and a binary COPY writer for the largest loads. Then it turns to the other large direction, reading, where a `Vec` of a million models is a million allocations held at once, and gives queries a stream of models that resolves its columns once and respects backpressure.

In this chapter you will learn:

- how `insert_many` sends a batch as one array per column through `unnest`
- why arrays of domain-typed columns need casts, and how shelf adds them
- how the derive writes a binary COPY row, and which types it must declare
- what each bulk strategy costs, measured against pgload's table
- how to stream models from a `RowStream`, resolving columns once
- how a COPY rejected at its start can kill a connection in 0.7.18, and how shelf avoids it

12.1 `insert_many`

Plan:

- One statement, `INSERT INTO "t" ("a", "b") SELECT * FROM unnest($1::text[], $2::bigint[])`, with one `Vec` per column built from the slice of `New…` values.
- In the select list, `unnest` keeps a composite whole; in `FROM` it expands it into fields (root PLAN.md §12 #4), which matters for composite-typed columns.
- Casts: an array of a domain needs the base type or a cast (root PLAN.md §12 #1); how the derive learns the SQL types (open question).

12.2 Binary COPY from the derive

Plan:

- The column types learnt from a prepared `SELECT "a", "b" FROM "t" LIMIT 0`, then a `BinaryCopyInWriter` fed one row per value through a generated method.
- The main book's warning: binary COPY trusts the declared types, and a wrong type of the same size loads wrong data silently (§12 #37).
- Pinned writer: scope it in a block; `drop` on a pinned reference drops nothing (§12 #34).

12.3 Measured

Plan:

- Ten thousand rows four ways through shelf: a loop of `insert`, pipelined `inserts`, `insert_many` in batches, binary COPY; times and ratios (to measure), set beside pgload's table (root PLAN.md §5.2; main book Chapter 10).
- Through lagproxy for the loop and the pipelined version, to show the round trips.

12.4 Streams of models

Plan:

- `query().stream(&db)` over `query_raw`: the columns resolved once from the first row (or the statement), each row decoded as it arrives.
- Backpressure: each request's response channel holds one message, so a slow consumer stalls the whole connection (main book Chapter 9); an unfinished stream stalls it too (§12 #35).
- Who owns the connection while the stream lives (root PLAN.md §12 #13), and whether shelf's stream should own a pooled one.

12.5 A COPY rejected at its start

Plan:

- In 0.7.18 a `COPY … FROM STDIN` that the server rejects before it starts (a missing table or column) leaves the connection unusable: rust-postgres issue #789 (root PLAN.md §12 #36).
- shelf's guard: the prepared `SELECT … LIMIT 0` of 12.2 fails first, cleanly, for a missing table or column.

The programs

- `run` (proposed): `insert_many` of a thousand rows, counted back.
- `run`: binary COPY of the same rows, with the types learnt from the table.
- `run`: four strategies timed on ten thousand rows; asserts COPY is faster than the loop by at least ten times (relative; to measure in debug).
- `run`: a stream of a hundred thousand models, counted without holding them.
- `run`: a missing column caught by the guard, and the connection still usable.

Side by side

Loading ten thousand rows: by hand (pgload's `unnest` batches) · shelf (`insert_many`, proposed) · sqlx (`QueryBuilder::push_values`, or `copy_in_raw`; check).

In stockroom

`stockroom seed` writes products and the ledger with binary COPY by hand (stockroom/src/cli/seed.rs).

Claims

- Measured: root PLAN.md §12 #22 (pgload); shelf's four strategies (to measure).
- Docs: rust-postgres issue #789; PostgreSQL's `COPY`.
- Opinion: offer `insert_many` by default, COPY when the batch is large and trusted.

When not to use shelf's bulk writes

- A client's CSV: COPY text through a staging table (main book Chapter 16), with errors reported by line.
- A load that needs `ON CONFLICT` per row: `insert_many` with a conflict clause, or `MERGE`.

Exercises

Plan:

1. Split `insert_many` into batches of a given size. *Hint:* chunks of the slice.
2. Make the stream report its row count at the end. *Hint:* `rows_affected()` once exhausted.
3. Load through COPY into a temporary table, then upsert. *Hint:* main book §10.6.

> **Open questions.** (1) How the derive learns the SQL types for `unnest` casts: from a prepared statement at run time, or from an attribute. (2) Whether shelf's stream owns its connection (root PLAN.md §12 #13). (3) The debug-build ratio for a relative timing assertion (to measure).

CHAPTER 13

Trusting the mapping: schema checks and tests

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, fourth chapter. Builds on Chapters 4 and 5, and on main book Chapter 4 (`Statement::columns`) and Chapter 17 (testing against a real database). shelf: `shelf::verify` and `Error::Schema` in `shelf/src/verify.rs` (proposed, S1c); the test suites (built, S1a): tests/expand.rs, tests/ui.rs, tests/from_row.rs and the harness tests/common/mod.rs. Crates: tokio-postgres 0.7.18, trybuild 1.0.122. Findings: PLAN.md §12 #6, #11, #12. Plan: orm/PLAN.md §6, §7, Chapter 13.

A struct and a table are two descriptions of the same data, kept in different places by different people. A migration renames a column; a column becomes nullable; a `bigint` becomes `numeric`. Chapter 5's check catches each of those, but only when the query runs, which may be the first request after a deploy, or a nightly job that runs at three in the morning.

This chapter moves the check earlier. `shelf::verify` prepares each model's own `SELECT` at startup, compares its columns with the struct's fields, asks the catalog which columns may be NULL, and reports every mismatch at once. Then it turns to the other side of trust, the macros themselves, and shows the three kinds of test shelf has used since phase S1a: expansion tests, UI tests and tests against the database, with what each one catches and what it cannot.

In this chapter you will learn:

- what can drift between a struct and its table, and when each kind of drift is caught
- how `verify` uses `Statement::columns()` and the catalog to check a model before it is used
- where to run it: at startup, and in a test per model
- how to test a derive: expansion tests, UI tests, integration tests
- what compile-time checking (sqlx's `query!`, Cornucopia) adds, and what it costs

13.1 What can drift

Plan:

- A renamed or dropped column, a changed type, a column made nullable, a new `NOT NULL` column a `New…` struct lacks: each with when it is caught today (Chapter 5: at the query; a NULL: at the row).

13.2 `verify`

Plan:

- Prepare the model's `SELECT` (no rows read): `resolve` already checks names and types.
- Nullability is not in a row description. `Column::table_oid()` and `column_id()` (built into 0.7.18; §12 #6) identify the table column, and `pg_attribute.attnotnull` says whether it can be NULL; a nullable column read into a non-`Option` field is reported.
- Every mismatch collected into `Error::Schema(Vec<Mismatch>)` (proposed).
- Computed columns and views have no table OID: `verify` says it cannot know.

13.3 Where to run it

Plan:

- At startup, before the server accepts requests, so a bad deploy fails its readiness check (main book Chapter 16).
- In a test per model against a migrated database (main book Chapter 17's template).

13.4 Testing a derive

Draft prose:

shelf tests its derives three ways, because each catches what the others cannot. An expansion test runs the generator on a struct and compares the pretty-printed code with a file: it catches any change to what users get, and its diff is the review. A UI test compiles a broken struct and compares the compiler's errors with a file: it pins down the message and where it points. An integration test derives for real and runs queries against PostgreSQL: it is the only one that proves the generated code does the right thing.

Plan:

- Excerpts: `tests/expand.rs` and one `.expanded.rs` file; `tests/ui.rs` and one `.stderr` file; one integration test with the harness.
- Blessing: `SHELF_BLESS=1` and `TRYBUILD=overwrite`, and why the diff must be read.

13.5 Compile-time checking

Plan:

- sqlx's `query!` checks SQL against a live database (or saved metadata in offline mode) at compile time (docs; check the current details); Cornucopia generates Rust from SQL (main book Chapter 14).
- What they buy (no drift reaches a build) and cost (a database at build time, or metadata to keep in sync); where `verify` sits between.

The programs

- `run` (proposed): `verify` on a model that matches its table.
- `run`: a renamed column, the report printed.
- `run`: a nullable column read into `String`, reported from the catalog.
- Excerpts of the test files named above, checked by `book.py excerpts`.

Side by side

Catching a renamed column: by hand (at the first query) · shelf (`verify` at startup, proposed) · sqlx (`query_as!` at compile time, check).

Claims

- Docs: PostgreSQL's `pg_attribute` catalog; sqlx's offline mode (check).
- Opinion: check the mapping at startup; a deploy that cannot run should fail before it takes traffic.

When not to use `verify`

- Queries written by hand: `verify` checks models' own SQL; a `FromRow` struct read from a custom query is checked when that query is prepared (Chapter 5).

Exercises

Plan:

1. Run `verify` for every model in one call. *Hint:* a list of functions, one per model.
2. Report extra columns the struct does not read. *Hint:* the statement has more columns than `resolve` used.
3. Write an expansion test for a struct of your own. *Hint:* add a file to `tests/expand/`.

> **Open questions.** (1) Views and computed columns: report "unknown" or skip. (2) The report format of `Error::Schema`. (3) Whether `verify` should also check `New…` structs against `NOT NULL` columns without defaults.

CHAPTER 14

Capstone: stockroom on shelf

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, fifth chapter. Builds on every chapter of this book, and on main book Chapter 18 (stockroom end to end) and stockroom's repository layer (stockroom/src/repo/). shelf: `orm/stockroom-shelf`, a library and binary reusing the `stockroom` crate's migrations and domain types (proposed, S2). Crates: axum 0.8.9, deadpool-postgres 0.14.2. Plan: orm/PLAN.md §4.1, §7, Chapter 14; §9, phase S2.

stockroom's data layer is written by hand: about a dozen repository functions, each with its SQL, its parameters and its `TryFrom<&Row>`. It is the code this book set out to generate. This chapter rewrites part of it with shelf, behind the same axum routes, against the same schema, with the same domain types, and compares the two line for line.

The comparison is honest in both directions. shelf removes the mapping code and most of the everyday SQL, and the routes keep their behaviour; the chapter counts lines, statements and round trips to show it. It also leaves alone the code shelf should not touch: the guarded `UPDATE` that keeps orders from overselling, the `MERGE` import, the COPY export, the `LISTEN` connection. Knowing where an ORM stops is the last thing this book teaches.

In this chapter you will learn:

- how shelf's models fit a real schema: a domain, a composite, an enum, a `jsonb` document, a virtual column
- what a repository function looks like rewritten with shelf, beside the original
- how much code, how many statements and how many round trips the rewrite changes, measured
- where shelf steps aside, and how raw SQL with `FromRow` fills the gap
- what a production ORM would add next

14.1 The shape

Plan:

- `orm/stockroom-shelf`: the `stockroom` crate as a dependency for its migrations, its domain types (`Sku`, `Address`, `OrderStatus`, `Cents`) and its error mapping; a repository layer on shelf; the routes for stores, products and orders, built with the same router shape.
- Its tests: stockroom's expectations for the ported routes, through `oneshot`, against a database cloned from stockroom's template (main book Chapter 17).

14.2 Models for a real schema

Plan:

- `Store` with an `address` composite (postgres-types' derive, passed through); `Product` with a `sku` domain (stockroom's hand-written `accepts`), `tags text[]`, `attributes jsonb` and a `version`; `StockLevel` with the virtual column `available`, read-only (never written by `Insert` or `Patch`: `generated`).
- `Order` and `OrderLine` with `has_many`, and `OrderStatus` as a Postgres enum.

14.3 Line for line

Plan:

- Three functions side by side: `stores::create`, `products::update` (versioned patch), and `orders::get` (an order with its lines).
- Counted (to measure): lines of Rust, statements prepared, round trips per request.

14.4 Where shelf steps aside

Plan:

- The oversell-proof reservation: `UPDATE stock … WHERE available >= $3`, pipelined in product-id order (main book Chapter 8): kept as SQL, its result read with a `FromRow` struct.
- The CSV import's `MERGE` and the CSV export's `COPY`, the `LISTEN` connection, the outbox worker's `SKIP LOCKED`: unchanged.
- The pattern: shelf for the everyday reads and writes, hand-written SQL for the statements whose wording is the point.

14.5 What a production ORM would add

Plan:

- Migrations generated from models, a query builder with joins, lazy relations, multi-database support, change tracking: what each would cost shelf in simplicity, and which existing crate already does it (diesel, SeaORM, toasty).

The programs

- `run` (proposed): the ported router on a pool of one, driven with `oneshot`: create a store, create a product, patch it with `If-Match`, place an order, read it back.
- `orm/stockroom-shelf/tests/`: stockroom's route tests for the ported routes.

Side by side

This chapter is one long side by side: stockroom's hand-written repository functions against the shelf versions, with the same routes on top.

The data

stockroom's schema, migrated by stockroom's own migrations, seeded with `stockroom seed`.

Claims

- Measured (to measure): lines, statements and round trips per ported route.
- Opinion: an ORM should cover the boring 80% and get out of the way of the rest.

When not to use shelf

- Every statement in 14.4, and any statement whose `WHERE` clause carries a business rule.

Exercises

Plan:

1. Port `GET /stores/{code}/stock`. *Hint:* the virtual column is read-only.
2. Add a route that lists a store's open orders with their lines. *Hint:* a filter, then a batched loader.
3. Count the round trips of `POST /orders` in both versions. *Hint:* `pg_stat_statements`, or lagproxy's timings.

> **Open questions.** (1) Which routes to port: stores, products and orders' reads are the proposal. (2) Whether stockroom-shelf reuses stockroom's test harness as a dependency or copies its pattern. (3) Whether `AppError` comes from stockroom unchanged, with `From<shelf::Error>`.

Appendix — attribute reference, generated code and phrasebook

> **Sketch, 7 November 2026.** A plan of this appendix, not its text; it will be replaced by the appendix. It collects reference material from every chapter: the `#[shelf(…)]` attributes (Chapters 4, 6, 8–11), what each derive generates (from the expansion tests in `orm/shelf-codegen/tests/expand/`), a phrasebook from shelf to diesel, SeaORM and sqlx (Chapter 0's comparison and each chapter's side by side), a glossary, and further reading. Entries already known are drafted; the rest is planned. Plan: orm/PLAN.md §4.3, §7, Appendix.

Look things up here once the chapters have taught them: which attribute does what and where it may go, what a derive writes for a given struct, and how to say a shelf call in the crate you use at work.

A Attribute reference

Drafted for the attributes that exist (phase S1a); the rest are added as their phases build them.

On the struct, read by `FromRow`:

Attribute	Meaning	Chapter
`rename_all = "…"`	Column names follow a convention: `"snake_case"` (the default), `"camelCase"`, `"PascalCase"` or `"SCREAMING_SNAKE_CASE"`	4

On a field, read by `FromRow`:

Attribute	Meaning	Chapter
`rename = "col"`	Read the column `col`; overrides `rename_all`	4
`skip`	Do not read; use `Default::default()`	4
`flatten`	A nested `FromRow` struct, read from the same row	4
`json`	Read a `json` or `jsonb` column through `Json<T>` (serde)	4
`try_from = Type`	Read a `Type`, then convert with `TryFrom`; the type may be quoted	4

Rules, as the derive enforces them: keys are refused if unknown or repeated; `skip` goes with nothing else; `flatten` goes with nothing but itself; `json` and `try_from` exclude each other; every error is reported at its token, all in one build.

Plan for the attributes still to come: `table`, `schema`, `primary_key`, `generated` (Chapter 6); `model` (Chapter 8); `version`, `updated_at` (Chapter 9); `belongs_to`, `has_many` (Chapter 11).

B What each derive generates

For this struct:

```
// orm/shelf-codegen/tests/expand/basic.rs (excerpt)
#[derive(FromRow)]
struct Product {
    id: i64,
    sku: String,
    r#type: String,
    tags: Option<Vec<String>>,
}
```

`#[derive(FromRow)]` generates, as the expansion test pins it down:

```
// orm/shelf-codegen/tests/expand/basic.expanded.rs (excerpt)
#[automatically_derived]
impl ::shelf::FromRow for Product {
    type Columns = (usize, usize, usize, usize);
    fn resolve(
        columns: &[::shelf::__private::tokio_postgres::Column],
    ) -> ::core::result::Result<Self::Columns, ::shelf::Error> {
        ::core::result::Result::Ok((
            ::shelf::__private::column::<i64>(columns, "Product", "id", "id")?,
            ::shelf::__private::column::<String>(columns, "Product", "sku", "sku")?,
            ::shelf::__private::column::<String>(columns, "Product", "type", "type")?,
            ::shelf::__private::column::<
                Option<Vec<String>>,
            >(columns, "Product", "tags", "tags")?,
        ))
    }
    fn from_row_at(
        row: &::shelf::__private::tokio_postgres::Row,
        columns: &Self::Columns,
    ) -> ::core::result::Result<Self, ::shelf::Error> {
        ::core::result::Result::Ok(Self {
            id: ::shelf::__private::get::<i64>(row, columns.0, "Product", "id")?,
            sku: ::shelf::__private::get::<String>(row, columns.1, "Product", "sku")?,
            r#type: ::shelf::__private::get::<
                String,
            >(row, columns.2, "Product", "type")?,
            tags: ::shelf::__private::get::<
                Option<Vec<String>>,
            >(row, columns.3, "Product", "tags")?,
        })
    }
}
```

Plan:

- The other expansion tests (`rename`, `skip_flatten`, `json_try_from`, `generics`) in the same form, each one line per attribute in the text and the full expansion as an excerpt.
- `Table`, `Insert` and `Patch` as their phases add them.

C Phrasebook

Plan: one table per task (map a row, find by key, insert, partial update, filter a page, load children, bulk insert), with the shelf call and the diesel, SeaORM and sqlx equivalents, each checked against the crate's version at the time of writing (check). A first row, to show the shape:

Task	shelf	sqlx	SeaORM
Read rows into a struct	`#[derive(FromRow)]`, `fetch_all`	`#[derive(FromRow)]`, `query_as`	`#[derive(DeriveEntityModel)]` (check)

D Glossary

Draft entries:

- **Derive macro**: a procedural macro attached with `#[derive(…)]`; it receives the item and adds code next to it (Chapter 3).
- **Expansion**: the code a macro produces in place of its invocation, or beside it for a derive; shelf prints it with `pretty` (Chapter 3).
- **Helper attribute**: an attribute a derive declares (`attributes(shelf)`) so the compiler passes it through to the derive (Chapter 4).
- **Hygiene**: the rules that keep a macro's names from clashing with the caller's; paths are not covered, hence the absolute paths in generated code (Chapters 2 and 3).
- **Resolve**: shelf's step that finds and checks each field's column once per statement (Chapter 5).
- **Span**: where a token came from; errors are reported at spans (Chapter 4).
- **TokenStream**: the sequence of tokens a procedural macro receives and returns; `proc_macro2::TokenStream` is the same thing usable outside the compiler (Chapter 3).
- **N+1**: one query for a list, then one per item (Chapter 11).

Plan: the terms of Part II and III as their chapters are written.

E Further reading

Plan:

- *The Little Book of Rust Macros*; the Rust Reference on macros by example and procedural macros.
- syn's and quote's documentation; dtolnay's proc-macro workshop.
- diesel's, SeaORM's and toasty's guides; the main book's Appendix F.

> **Open questions.** (1) Whether the phrasebook belongs here or in Chapter 0, with a short version in the other. (2) Whether B should include every expansion test or one per derive.